REINFORCEMENT LEARNING

Homework 2 (Unit 5) - Project Manual

Project 7: Self-Driving Lab Simulator

Q-learning agent that chooses temperature and concentration to maximise reaction yield

Name	Saniya Kousar
Branch	Computer Science Engineering(BE)
Roll No.	100523733092
Subject	Reinforcement Learning
Marks	10

Contents

1. Aim
2. Software and tools
3. Theory
4. Problem formulation
5. Algorithm (step by step procedure)
6. Program (source code)
7. Output (results, learning curves, tables)
8. Result and conclusion
9. Limitations
10. Ethics and safety
11. Viva questions

1. Aim

To build a simulated self-driving laboratory in which a reinforcement learning agent (tabular Q-learning) learns to select the temperature and concentration of a chemical reaction so as to maximise the yield within a fixed experiment budget while avoiding an unsafe temperature zone, and to compare it with random and heuristic baselines.

2. Software and tools

- Google Colab (Python 3, no GPU required)
- NumPy - environment and Q-table
- Matplotlib - learning curves, heat maps, policy map
- Pandas - results table

3. Theory

Reinforcement learning is learning by trial and error. An agent observes a state s, takes an action a, receives a reward r and moves to a new state s'. The aim is to learn a policy that maximises the expected sum of discounted rewards.

Q-learning is an off-policy, model-free algorithm that learns the action-value table Q(s,a) using the update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$$

where alpha is the learning rate and gamma the discount factor. An epsilon-greedy rule balances exploration (random action with probability epsilon) and exploitation (best known action). Epsilon decays from 1.0 to 0.05 during training.

In a self-driving lab each experiment is expensive, so choosing the next experiment well is a sequential decision problem - which fits RL naturally.

4. Problem formulation

Element	Definition
Environment	Simulated reaction. True yield (0-100%) depends on temperature T (30-120 C, step 10) and concentration C (0.1-1.0 mol/L, step 0.1). Global optimum 95% at 80 C / 0.6 M; decoy local optimum near 45 C / 0.9 M; decomposition above 100 C; measurement noise SD 2%.
State	Current recipe (T index, C index): 10 x 10 = 100 states.
Actions	4: T+10, T-10, C+0.1, C-0.1. Each action runs one experiment at the new recipe.
Reward	measured yield/100 - 0.01 (experiment cost) - 0.3 if T >= 110 C (unsafe).
Episode	25 experiments from a random starting recipe.
Algorithm	Tabular Q-learning, alpha 0.05, gamma 0.9, epsilon 1.0 to 0.05, 6000 episodes, 5 seeds.
Baselines	Random experiments; hill-climbing heuristic (keep last move while yield improves, else random move).

5. Algorithm (step by step procedure)

1. Create the simulated lab environment with the hidden yield function.
2. Initialise the Q-table (100 states x 4 actions) with zeros.
3. For each training episode: reset the lab to a random recipe.

4. Repeat for 25 experiments: choose an action by epsilon-greedy, run the experiment, observe the reward and next state.
5. Update Q(s,a) with the Q-learning rule and move to the next state.
6. Reduce epsilon gradually; every 100 episodes evaluate the greedy policy (no exploration).
7. Repeat for 5 random seeds and average the learning curves.
8. Evaluate random and heuristic baselines with the same protocol and compare.

6. Program (source code)

Cell 1 - Environment, agents and Q-learning training function

```
import numpy as np, matplotlib.pyplot as plt
%matplotlib inline

# ---------------- Environment ----------------
class SelfDrivingLabEnv:
    """Toy self-driving lab: agent tunes Temperature (T) and Concentration (C)."""
    T LEVELS = np.arange(30, 130, 10)          # 30..120 degC  (10 levels)
    C LEVELS = np.round(np.arange(0.1, 1.1, 0.1), 1)  # 0.1..1.0 mol/L (10 levels)
    ACTIONS = ["T+10", "T-10", "C+0.1", "C-0.1"]
    UNSAFE T = 110                              # runaway-reaction zone
    def __init__(self, budget=25, noise sd=2.0, seed=0):
        self.nT, self.nC = len(self.T LEVELS), len(self.C LEVELS)
        self.budget, self.noise sd = budget, noise sd
        self.rng = np.random.default rng(seed)
        self.n states, self.n actions = self.nT*self.nC, 4
        # precompute noise-free yield table (percent)
        self.Y = np.array([[self.true yield(t, c) for c in self.C LEVELS] for t in
self.T LEVELS])
    @staticmethod
    def true yield(T, C):
        main  = 95*np.exp(-((T-80)/18)**2 - ((C-0.6)/0.25)**2)     # global optimum 80C, 0.6 M
        decoy = 55*np.exp(-((T-45)/10)**2 - ((C-0.9)/0.12)**2)     # misleading local optimum
        decomposition = np.exp(-max(0, T-100)/12)                  # product breaks down when too
hot
        return (main+decoy)*decomposition
    def reset(self):
        self.i, self.j = self.rng.integers(self.nT), self.rng.integers(self.nC)
        self.t = 0
        return self.i*self.nC + self.j
    def step(self, a):
        if a == 0: self.i = min(self.i+1, self.nT-1)
        if a == 1: self.i = max(self.i-1, 0)
        if a == 2: self.j = min(self.j+1, self.nC-1)
        if a == 3: self.j = max(self.j-1, 0)
        self.t += 1
        measured = np.clip(self.Y[self.i, self.j] + self.rng.normal(0, self.noise sd), 0, 100)
        unsafe = self.T LEVELS[self.i] >= self.UNSAFE T
        reward = measured/100 - 0.01 - (0.3 if unsafe else 0.0)   # yield - experiment cost -
safety penalty
        done = self.t >= self.budget
        return self.i*self.nC + self.j, reward, done, dict(true yield=self.Y[self.i, self.j],
unsafe=unsafe)

# ---------------- Agents ----------------
def run policy(env, policy fn, n eval=300):
    """Returns mean true yield over the episode, mean final yield, best-found yield, unsafe-
experiment rate."""
    avg, fin, best, uns = [], [], [], []
    for _ in range(n eval):
        s = env.reset(); ys, us = [], 0; last a, last y = None, -1
        done = False
        while not done:
            a = policy fn(s, last a, last y)
            s, r, done, info = env.step(a)
            ys.append(info["true yield"]); us += info["unsafe"]
            last a, last y = a, info["true yield"]
        avg.append(np.mean(ys)); fin.append(ys[-1]); best.append(np.max(ys));
uns.append(us/len(ys))
    return np.mean(avg), np.mean(fin), np.mean(best), np.mean(uns)

def make random(env):
    return lambda s, la, ly: env.rng.integers(4)
def make hillclimb(env):
    """Human-like heuristic: keep going the same way while yield improves, otherwise try a random
new direction."""
    state = {"prev": -1}
    def pol(s, la, ly):
        if la is None: state["prev"] = -1; return env.rng.integers(4)
        improved = ly > state["prev"]; state["prev"] = ly
        return la if improved else env.rng.integers(4)
    return pol
def make greedy(Q):
    return lambda s, la, ly: int(np.argmax(Q[s]))

def train qlearning(episodes=3000, alpha=0.1, gamma=0.95, eps start=1.0, eps end=0.05, seed=0,
eval every=100, n eval=100):
    env = SelfDrivingLabEnv(seed=seed); eval env = SelfDrivingLabEnv(seed=1000+seed)
    Q = np.zeros((env.n states, env.n actions)); rng = np.random.default rng(seed)
    curve, ep returns = [], []
    for ep in range(episodes):
        eps = max(eps end, eps start - (eps start-eps end)*ep/(0.7*episodes))
        s, done, G = env.reset(), False, 0
        while not done:
```

```
            a = rng.integers(4) if rng.random() < eps else int(np.argmax(Q[s]))
            s2, r, done,   = env.step(a)
            target = r + (0 if done else gamma*np.max(Q[s2]))
            Q[s, a] += alpha*(target - Q[s, a]); s = s2; G += r
        ep_returns.append(G)
        if (ep+1) % eval_every == 0:
            curve.append(run_policy(eval_env, make_greedy(Q), n_eval)[0])
    return Q, np.array(curve), np.array(ep_returns)
```

Cell 2 - Plot the hidden reaction landscape

```
env = SelfDrivingLabEnv(seed=0)
plt.figure(figsize=(6,4.5))
plt.imshow(env.Y, origin="lower", cmap="viridis", aspect="auto",
           extent=[env.C_LEVELS[0]-.05, env.C_LEVELS[-1]+.05, env.T_LEVELS[0]-5, env.T_LEVELS[-
1]+5])
plt.colorbar(label="True yield (%)")
plt.axhline(env.UNSAFE_T-5, color="red", ls="--", label="unsafe zone (T >= 110 C)")
plt.xlabel("Concentration (mol/L)"); plt.ylabel("Temperature (C)"); plt.title("Reaction yield
landscape"); plt.legend()
plt.savefig("fig1_landscape.png", dpi=200, bbox_inches="tight"); plt.show()
print("Best possible yield: %.1f%% at T=%d C, C=%.1f M" % (env.Y.max(),
env.T_LEVELS[np.unravel_index(env.Y.argmax(), env.Y.shape)[0]],
env.C_LEVELS[np.unravel_index(env.Y.argmax(), env.Y.shape)[1]]))
```

Cell 3 - Train Q-learning (5 seeds) and evaluate baselines

```
N_SEEDS, EPISODES, EVAL_EVERY = 5, 6000, 100
curves, Qs = [], []
for seed in range(N_SEEDS):
    Q, curve,   = train_qlearning(episodes=EPISODES, alpha=0.05, gamma=0.9, seed=seed,
eval_every=EVAL_EVERY, n_eval=200)
    curves.append(curve); Qs.append(Q); print("seed", seed, "final avg yield %.1f%%" % curve[-
5:].mean())
curves = np.array(curves)

# baselines (evaluated with the same protocol: 25 experiments, random starts, 2000 episodes)
base_env = SelfDrivingLabEnv(seed=123)
rand_res = run_policy(base_env, make_random(base_env), 2000)
hill_res = run_policy(base_env, make_hillclimb(base_env), 2000)
q_res = np.mean([run_policy(SelfDrivingLabEnv(seed=500+i), make_greedy(Qs[i]), 2000) for i in
range(N_SEEDS)], axis=0)
```

Cell 4 - Learning curve

```
x = np.arange(1, curves.shape[1]+1) * EVAL_EVERY
m, s = curves.mean(0), curves.std(0)
plt.figure(figsize=(7,4.5))
plt.plot(x, m, label="Q-learning (mean of 5 seeds)", color="tab:green"); plt.fill_between(x, m-s,
m+s, alpha=0.25, color="tab:green")
plt.axhline(rand_res[0], color="tab:red", ls="--", label="Random baseline")
plt.axhline(hill_res[0], color="tab:orange", ls="--", label="Hill-climb heuristic")
plt.axhline(env.Y.max(), color="gray", ls=":", label="Theoretical max yield")
plt.xlabel("Training episodes"); plt.ylabel("Avg true yield per experiment (%)")
plt.title("Learning curve: self-driving lab"); plt.legend(); plt.grid(alpha=.3)
plt.savefig("fig2_learning_curve.png", dpi=200, bbox_inches="tight"); plt.show()
```

Cell 5 - Results table and bar chart

```
import pandas as pd
df = pd.DataFrame({"Method": ["Random", "Hill-climb heuristic", "Q-learning (RL)"],
    "Avg yield per experiment (%)": [rand_res[0], hill_res[0], q_res[0]],
    "Final-experiment yield (%)": [rand_res[1], hill_res[1], q_res[1]],
    "Best yield found (%)": [rand_res[2], hill_res[2], q_res[2]],
    "Unsafe experiments (%)": [100*rand_res[3], 100*hill_res[3], 100*q_res[3]]}).round(1)
display(df); df.to_csv("results_table.csv", index=False)
ax = df.set_index("Method")[["Avg yield per experiment (%)", "Final-experiment yield (%)", "Best
yield found (%)"]].plot.bar(figsize=(7,4.5), rot=0)
ax.set_ylabel("Yield (%)"); ax.set_title("RL vs baselines"); ax.grid(axis="y", alpha=.3)
plt.savefig("fig3_comparison.png", dpi=200, bbox_inches="tight"); plt.show()
```

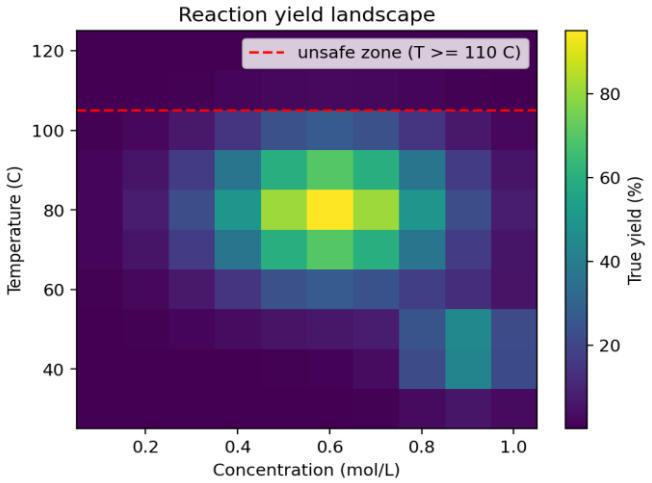
Cell 6 - Learned policy map and an example experiment run

```
Q = Qs[0]; arrows = {0: (0, 1), 1: (0, -1), 2: (1, 0), 3: (-1, 0)}   # (dx=conc, dy=temp)
fig, ax = plt.subplots(figsize=(7,5.5))
ax.imshow(env.Y, origin="lower", cmap="viridis", aspect="auto", extent=[-.5, 9.5, -.5, 9.5])
for i in range(env.nT):
    for j in range(env.nC):
        dx, dy = arrows[int(np.argmax(Q[i*env.nC+j]))]
        ax.arrow(j-dx*.25, i-dy*.25, dx*.5, dy*.5, head_width=.18, color="white")
```

```
ax.set xticks(range(10)); ax.set xticklabels(env.C LEVELS); ax.set yticks(range(10));
ax.set yticklabels(env.T LEVELS)
ax.set xlabel("Concentration (mol/L)"); ax.set ylabel("Temperature (C)"); ax.set title("Learned
policy (arrows = chosen action)")
plt.savefig("fig4 policy.png", dpi=200, bbox inches="tight"); plt.show()

# one example experiment sequence
e = SelfDrivingLabEnv(seed=7); s = e.reset(); path = [(e.T LEVELS[e.i], e.C LEVELS[e.j], e.Y[e.i,
e.j])]; done = False
while not done:
    s, r, done, info = e.step(int(np.argmax(Q[s]))); path.append((e.T LEVELS[e.i],
e.C LEVELS[e.j], info["true yield"]))
print("Experiment # | Temp (C) | Conc (M) | True yield (%)")
for k, (t, c, y) in enumerate(path): print("%12d | %8d | %8.1f | %14.1f" % (k, t, c, y))
```

7. Output



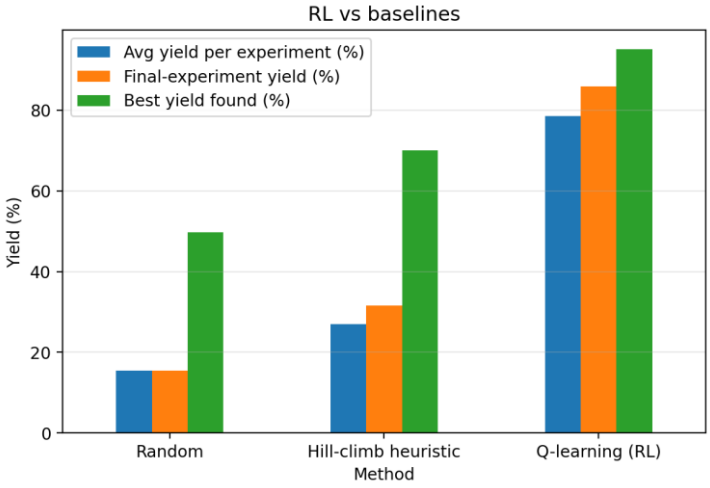
Output 1: Hidden reaction landscape (agent only sees noisy yields).



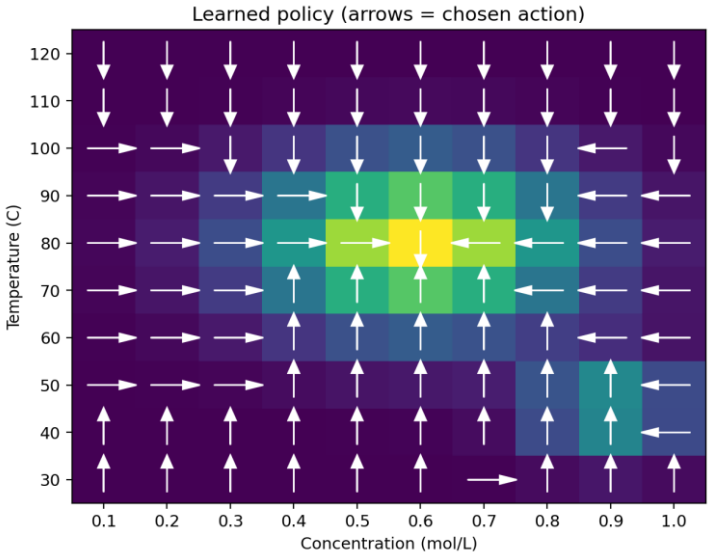
Output 2: Learning curve of Q-learning (mean +/- SD of 5 seeds) vs baselines.

Output 3: Results table (2000 evaluation episodes)

Method	Avg yield per experiment (%)	Final-experiment yield (%)	Best yield found (%)	Unsafe experiments (%)
Random	15.5	15.4	49.7	20.1
Hill-climb heuristic	27.0	31.6	70.0	11.5
Q-learning (RL)	78.5	85.8	95.0	0.4



Output 4: RL vs baselines.



Output 5: Learned policy - arrows lead from every recipe to the high-yield region.

Output 6: Example experiment sequence from a bad start (120 C, 0.7 M)

Experiment	0	1	2	3	4	5
Temp (C)	120	110	100	90	80	80
Conc (M)	0.7	0.7	0.7	0.7	0.7	0.6
True yield (%)	0.1	2.2	23.6	59.5	81.0	95.0

8. Result and conclusion

- Q-learning raised the average yield per experiment from 15.5% (random) and 27.0% (heuristic) to 78.5%.
- The agent found the 95% optimum in every episode and avoided the decoy local optimum.
- Unsafe experiments fell from 20.1% (random) to 0.4%, showing it learned the safety constraint through the reward penalty.

Hence reinforcement learning can plan experiments efficiently in a simulated self-driving lab and clearly outperforms random and heuristic selection.

9. Limitations

- Hand-made simulated yield function; real chemistry has many more variables (pH, time, catalyst, mixing).
- Tabular Q-learning only suits a small discrete grid; larger problems need DQN, PPO or Bayesian optimisation.
- The policy is specific to one reaction and needs thousands of simulated episodes.
- The state does not include the remaining budget, so the agent sometimes oscillates between 80 C and 70 C near the end of an episode.

10. Ethics and safety

- Physical safety: unsafe exploration in a real lab can cause fires or toxic releases; train in simulation first, add hard safety limits and human approval.
- Accountability and reproducibility: log every experiment; scientists remain responsible for conclusions.
- Impact on people and access: automation should assist technicians and be available to smaller institutions.